

C++ Programming Textbook Notes

Detailed Guide: Arrays (1D, 2D, 3D, Pointer Math & Modern STL Containers) in C++

1. Overview of Arrays in C++

Conceptual Explanation

An **Array** is a linear collection of elements of the same data type stored in strictly contiguous (continuous) physical memory locations. An array variable is also historically referred to as a subscripted variable.

Why Do We Need Arrays? Suppose you need to write a program to process the exam scores or average values of 100 students:

- **The Regular Method:** You would have to declare 100 separate scalar variables (`int n1, n2, n3... n100;`). Reading, updating, and passing 100 distinct variable names makes code unmaintainable, error-prone, and impossible to iterate over cleanly.
- **The Array Method:** You declare a single contiguous array variable of size 100 (`int numbers[100];`). Every element shares the same base variable name and is accessed efficiently using an integer index offset.

Memory Architecture & Pointer Decay

In C and C++, array elements reside side-by-side in RAM. The array variable name itself acts as a constant pointer holding the base address of the very first element (index 0). Accessing `a[i]` is evaluated by the compiler as direct pointer arithmetic:

Pointer Arithmetic Formula:

Address of Element `a[i]` = Base Address + (`i * sizeof(data_type)`)

RAM Memory Layout of `int a[5] = {10, 50, 30, 70, 20}` (Base Address = `0x1000`):

Index:	<code>a[0]</code>	<code>a[1]</code>	<code>a[2]</code>	<code>a[3]</code>	<code>a[4]</code>
Value:	10	50	30	70	20
RAM Address:	<code>0x1000</code> (Base)	<code>0x1004</code> (+4 Bytes)	<code>0x1008</code> (+8 Bytes)	<code>0x100C</code> (+12 Bytes)	<code>0x1010</code> (+16 Bytes)

2. One-Dimensional (1-D) Arrays in C++

A. Declaration Syntax & Subscript Operator

To declare a 1-D array, specify the data type, an identifier name, and a size constant inside square brackets:

```
data_type array_name[size]; // Example: int age[100];
```

Indices always run from 0 up to `size - 1`. In `age[3] = 26;`, square brackets `[]` act as the subscript operator, assigning 26 to the 4th element slot.

B. Critical Rules for 1-D Array Declarations

1. **Size Cannot Be Omitted in Standalone Declarations:** Writing `int a[];` without immediate initialization triggers a compilation error.

2. **Contiguous Memory Guarantee:** Arrays always allocate a single unbroken block of RAM.
3. **The Garbage Value Trap:** Local raw arrays declared inside functions without initialization contain unpredictable, uninitialized memory bits ("garbage values").
4. **Complete Initialization:** Initializing explicit values: `int a[5] = {10, 50, 30, 70, 20};`.
5. **Array Overflow:** Providing more initializers than the explicitly stated size causes a compilation error (e.g., `int a[5] = {10, 20, 30, 40, 50, 60};` ❌ Error).
6. **Partial Initialization Rule:** Supplying fewer initializers than declared size causes the compiler to automatically zero-fill remaining slots (e.g., `int a[5] = {10, 50};` results in `[10, 50, 0, 0, 0]`).
7. **Implicit Sizing:** Omitting size brackets is legal *only* when initializers are supplied concurrently (e.g., `int a[] = {10, 20, 30};` creates size 3).
8. **Modern C++ Value Initialization:** In C++11, writing `int a[5]{};` zero-initializes all 5 array elements cleanly.

1-D Array Code Example

```
#include <iostream>

int main() {
    // Complete initialization
    int scores[5] = {10, 50, 30, 70, 20};

    // Partial initialization: remaining 3 slots populated automatically with 0
    int partial[5] = {10, 50};

    // Modern C++ uniform zero-initialization
    int zeroFilled[5]{};

    std::cout << "Partial Array Slot [2]: " << partial[2] << "
"; // Outputs 0
    std::cout << "Zero Filled Slot [0]: " << zeroFilled[0] << "
"; // Outputs 0

    return 0;
}
```

⚠️ **Out-of-Bounds Indexing Bug:** C++ does not perform automatic runtime bounds checking on raw arrays. Reading or writing past limits (e.g., accessing `a[10]` on size 5 array) corrupts adjacent stack memory or triggers segmentation faults.

3. Two-Dimensional (2-D) Arrays (Matrices)

Conceptual Explanation

A Two-Dimensional Array is structurally an "Array of Arrays". It maps data visually into a grid matrix composed of horizontal rows and vertical columns.

A. Syntax & Initializations

```
data_type array_name[rows][columns]; // Total capacity = rows * columns
```

Example: `int arr[3][2];` allocates a 3-row by 2-column matrix (6 elements total).

B. Structural Rules for 2-D Initialization

When initializing a 2-D array during declaration, specifying the second dimension (columns) is **strictly mandatory**. The row dimension is optional:

Variant Status	Syntax Expression	Compiler Interpretation
Correct	<code>int arr[2][3] = {{5, 8, 9}, {10, 12, 11}};</code>	Explicit 2 rows, 3 columns.
Correct	<code>int arr[][3] = {12, 34, 56, 78, 91, 23};</code>	Infers 2 rows based on 6 items / 3 cols.
Incorrect ❌	<code>int arr[2][];</code>	Compilation Error: Column count missing!
Incorrect ❌	<code>int arr[][];</code>	Compilation Error: Dimensions incomplete!

C. Memory Layout: Row-Major Order

Although conceptualized as a 2D grid, physical RAM is strictly a 1D linear memory address space. C++ stores 2-D arrays using **Row-Major Order**: Row 0 is stored completely, immediately followed by Row 1, and so on.

Grid View of `int a[2][3]`:

	Col 0	Col 1	Col 2
Row 0	5	8	9
Row 1	10	12	11

Row-Major Physical RAM Layout:

Index:	[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]
Value:	5	8	9	10	12	11
RAM Addr:	0x1000	0x1004	0x1008	0x100C	0x1010	0x1014

Implementation Blueprint: Printing Matrix in C++

```
#include <iostream>

int main() {
    // Declare 2-D matrix with 2 rows and 3 columns
    int a[2][3] = {
        {5, 8, 9},
        {10, 12, 11}
    };

    // Nested loops for Matrix Navigation
    for (int i = 0; i < 2; i++) { // Outer loop controls Row index
        for (int j = 0; j < 3; j++) { // Inner loop controls Column index
            std::cout << a[i][j] << " ";
        }
        std::cout << "\n"; // Newline after completing row
    }
}
```

```
    return 0;
}
```

4. Three-Dimensional (3-D) Arrays

Conceptual Explanation

A Three-Dimensional Array is an "Array of 2-D Arrays". Think of it as a multi-page collection of spreadsheets, where each page represents an independent 2-D matrix grid.

```
data_type array_name[pages][rows][columns];
```

Example: `int a[2][3][2];` allocates 2 page blocks, each containing 3 rows and 2 columns (12 integers total).

Implementation Blueprint: Accessing 3-D Space

```
#include <iostream>

int main() {
    int a[2][3][2] = {
        { {4, 5}, {6, 7}, {8, 0} }, // Matrix Page 0
        { {11, 12}, {13, 0}, {0, 0} } // Matrix Page 1
    };

    // Three nested loops to iterate over 3-D space
    for (int i = 0; i < 2; i++) { // Loop 1: Page Layers
        for (int j = 0; j < 3; j++) { // Loop 2: Rows on Current Page
            for (int k = 0; k < 2; k++) { // Loop 3: Columns in Current Row
                std::cout << a[i][j][k] << " ";
            }
            std::cout << "\n";
        }
        std::cout << "\n"; // Extra blank line after completing a full page block
    }

    return 0;
}
```

5. Modern C++ Array Alternatives (`std::array` & `std::vector`)

Conceptual Explanation

While C-style raw arrays are fundamental, C++ introduces type-safe, object-oriented standard library alternatives in the header files `<array>` and `<vector>`:

- `std::array<T, N>` (Fixed-size Stack Array): Wraps raw arrays without performance loss. Knows its own size (`.size()`), does not decay to raw pointers implicitly, and supports safe bounds-checked element access via `.at(index)`.

- `std::vector<T>` (Dynamic Heap Array): Automatically expands or shrinks at runtime as elements are inserted or deleted.

Comparative Matrix: Raw C-Array vs `std::array` vs `std::vector`

Feature	C-Style Raw Array	<code>std::array<T, N></code>	<code>std::vector<T></code>
Memory Location	Stack	Stack	Dynamic Heap
Sizing Mutability	Fixed compile-time	Fixed compile-time	Dynamic runtime resizing
Knows Its Size	✗ No (Pointer Decay)	✓ Yes (<code>.size()</code>)	✓ Yes (<code>.size()</code>)
Safe Access	✗ No bounds check	✓ Yes (via <code>.at()</code>)	✓ Yes (via <code>.at()</code>)

Modern C++ Code Demonstration

```
#include <iostream>
#include <array>
#include <vector>

int main() {
    // 1. Modern C++ Fixed Stack Array
    std::array<int, 5> modernArr = {10, 20, 30, 40, 50};
    std::cout << "std::array size: " << modernArr.size() << "
";
    std::cout << "Safe element access: " << modernArr.at(2) << "
";

    // 2. Modern C++ Dynamic Heap Vector
    std::vector<int> dynamicVec = {100, 200};
    dynamicVec.push_back(300); // Dynamically expands size at runtime

    // Range-based for loop iteration
    std::cout << "Vector items: ";
    for (int val : dynamicVec) {
        std::cout << val << " ";
    }
    std::cout << "
";

    return 0;
}
```

💡 **Modern C++ Best Practice:** In modern C++, avoid using raw C-style arrays for new application logic. Prefer `std::array` for fixed stack buffers, and `std::vector` for dynamic collections.

6. Summary Dimensional Reference

Dimension	Index Reference	Access Analogy	Structural Purpose
1-D Array	<code>a[i]</code>	Single row of post office boxes	Linear tracking lists

Dimension	Index Reference	Access Analogy	Structural Purpose
2-D Array	a[i][j]	A flat spreadsheet page grid	Mathematical matrix computation
3-D Array	a[i][j][k]	Multi-page collection of spreadsheets	Complex spatial data tracking